

Day 44: BankAccount Constructor

Day 44 — BankAccount Constructor

1. Day Number

Day 44

2. Topic Name

Constructor `__init__()`

Today you will learn how a Python class can automatically receive and store data when an object is created.

Main idea:

```
Class
↓
Constructor __init__()
↓
Initialize object data
↓
Object ready to use
```

3. Connection

Yesterday, you created a **User class** and learned about:

- class
- object
- attributes

Today, you will create a **BankAccount class**.

Instead of setting attributes separately after creating an object, you will use a constructor to provide the values immediately.

Conceptually:

```
Yesterday

User class
↓
Create object
↓
Store user information
```

Today:

```
BankAccount class
↓
__init__()
```

↓
Receive holder name + balance
↓
Create initialized account object

4. Important Topics

Today's important topics are:

- class
- constructor
- `__init__()`
- `self`
- attributes
- object creation

Basic structure:

```
class ClassName:  
    def __init__(self, value):  
        self.value = value
```

Do not worry if this looks unfamiliar. We will break it down.

5. Foundational Notes

What is a class?

A **class** is like a blueprint.

For example:

```
BankAccount blueprint  
  
Account holder  
Balance
```

The class describes what information every bank account object should have.

What is an object?

An **object** is an actual item created using the class.

For example:

```
BankAccount class  
├── Account object for Amit  
└── Account object for Neha
```

Each object can have different values.

What is a constructor?

A constructor is a special method that runs automatically when an object is created.

In Python, the constructor is usually:

```
__init__()
```

Example idea:

```
Create object
    ↓
Python calls __init__()
    ↓
Values are stored
    ↓
Object is initialized
```

You do **not** normally call `__init__()` yourself.

Python calls it automatically.

What does `__init__` mean?

You can think of it as:

```
initialize this new object
```

For example:

```
def __init__(self, name):
```

This means the constructor receives a `name` when the object is created.

What is `self`?

`self` represents the **current object**.

Suppose you have:

```
self.name = name
```

The right side:

```
name
```

is the value received by the constructor.

The left side:

```
self.name
```

creates an attribute belonging to the object.

Think of it as:

```
received value
  ↓
  name
  ↓
store inside current object
  ↓
self.name
```

Constructor parameter vs object attribute

This is an important difference.

Consider:

```
def __init__(self, name):
    self.name = name
```

Here:

```
name
```

is a **parameter**.

But:

```
self.name
```

is an **object attribute**.

Conceptually:

```
Input value
  ↓
parameter: name
  ↓
self.name
  ↓
stored inside object
```

Why use a constructor?

Without a constructor, you may create an object first and then manually assign values.

Conceptually:

```
Create object
↓
set name
```

```
↓  
set balance
```

With a constructor:

```
Create object(name, balance)  
    ↓  
    __init__()  
    ↓  
everything initialized immediately
```

This is cleaner and more organized.

6. Easy Example

Let's use a different example so that we don't solve today's BankAccount problem for you.

Suppose we have a Book class:

```
class Book:  
    def __init__(self, title, price):  
        self.title = title  
        self.price = price
```

Create an object:

```
book1 = Book("Python Basics", 300)
```

Now conceptually:

```
Book("Python Basics", 300)  
    ↓  
    __init__()  
    |  
    ├── title = "Python Basics"  
    └── price = 300  
    ↓  
    book1
```

We could access an attribute using:

```
print(book1.title)
```

Output:

```
Python Basics
```

Notice that we did not manually call:

```
__init__()
```

Python called it when this happened:

```
Book(...)
```

7. Problem Statement

Create a class called:

```
BankAccount
```

The class should use a constructor to store:

- account holder name
- balance

Create **one bank account object**.

For example, conceptually your object might represent:

```
Account holder: Amit  
Balance: 5000
```

Your goal is to practice:

```
class  
  ↓  
__init__()  
  ↓  
self  
  ↓  
attributes  
  ↓  
object
```

Do not add deposit or withdrawal functionality yet.

Keep today's problem focused only on creating and initializing the object.

8. Concepts Used

You will use:

Class

Defines the structure of a bank account.

```
BankAccount
```

Constructor

Initializes the object.

```
__init__()
```

Parameters

Receive values such as:

```
holder name  
balance
```

self

Represents the current bank account object.

Attributes

Store information inside the object.

Conceptually:

```
self.account_holder  
self.balance
```

Object

An actual bank account created from the class.

9. Thought Process

Before writing code, think through the problem.

Step 1: What object am I modelling?

A:

```
Bank Account
```

So you need a:

```
BankAccount class
```

Step 2: What information should each bank account contain?

The problem asks for:

```
account holder name  
balance
```

Therefore, these should become attributes.

Step 3: When should these values be assigned?

Immediately when the object is created.

Therefore, use:

```
__init__()
```

Step 4: What should the constructor receive?

Conceptually:

```
self  
account holder name  
balance
```

Remember:

```
self
```

is supplied automatically by Python when working with the object.

You provide the other values.

Step 5: Where should the received values be stored?

Inside the object:

```
self.<attribute>
```

Step 6: Create one object

Pass an account holder name and starting balance.

Flow:

```
BankAccount class  
  ↓  
  __init__()  
  ↓  
  holder + balance received  
  ↓  
  attributes created  
  ↓  
  BankAccount object
```

10. Pseudocode

Do not write Python immediately.

Start with this logic:

```
CREATE a BankAccount class  
  
  CREATE a constructor  
  
    RECEIVE account holder name  
    RECEIVE balance
```

```
STORE account holder name in the object
STORE balance in the object
```

```
CREATE one BankAccount object
  PASS a holder name
  PASS a balance
```

```
PRINT or inspect the object's information
```

More visually:

```
START
|
v
Define BankAccount class
|
v
Define constructor
|
+--> receive holder name
|
+--> receive balance
|
v
Store values as attributes
|
v
Create BankAccount object
|
v
Access object attributes
|
v
END
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Your program structure should roughly be:

```
Class Definition
|
v
Constructor
|
v
Object Attributes
|
v
Create Object
|
```

v
Use Object

Avoid unnecessary things today such as:

```
deposit()  
withdraw()  
transaction history  
multiple accounts  
inheritance  
file handling
```

Those can come later.

Today your goal is simply understanding:

```
How does __init__() initialize an object?
```

12. Easy Edge Cases

Edge Case 1: Balance is 0

Example:

```
Account holder: Amit  
Balance: 0
```

This can be perfectly valid.

A newly created bank account may have a zero balance.

So don't assume:

```
0 = invalid
```

for today's simple exercise.

Edge Case 2: Empty Account Holder Name

Example:

```
Account holder: ""  
Balance: 5000
```

For today's beginner exercise, simply notice that this is possible.

Later you can add validation such as:

```
if name is empty  
    reject account
```

But you don't need that additional logic yet unless you want extra practice.

13. Expected Input

No `input()` is required unless you want extra practice.

You can directly provide values while creating the object.

Conceptually:

```
Account holder name = "Amit"  
Balance = 5000
```

Then create:

```
BankAccount object  
    ↓  
"Amit", 5000
```

14. Expected Output

Your program should be able to create an object containing values similar to:

```
Account Holder: Amit  
Balance: 5000
```

The important result is that the object internally contains:

```
BankAccount object  
|  
├─ account holder → Amit  
|  
└─ balance → 5000
```

The exact print formatting is not important today.

The main learning goal is **constructor-based initialization**.

15. Hint Only

Start with this skeleton:

```
class BankAccount:  
    def __init__(self, ...):  
        ...
```

Ask yourself:

1. What two values should `__init__()` receive?
2. How do you convert those values into object attributes using `self`?
3. How do you create an object and pass those two values?

Remember the pattern from the Book example:

```
self.attribute = parameter
```

Your main mental model for Day 44 should be:

```
Class
↓
Object creation
↓
__init__() automatically runs
↓
Parameters receive values
↓
self stores those values
↓
Object now has attributes
```

Do not add methods like deposit or withdraw yet. Day 44 is about becoming comfortable with the constructor, `self`, and object initialization.